

Lecture 11: Kernels Part 2

*Date: February 20, 2025***Lecture Roadmap:**

1. **Recap:** Review the kernel trick and feature maps from last lecture
2. **Kernel Machines:** Learn how to "kernelize" common ML algorithms
 - Kernel Ridge Regression
 - Kernel SVM
 - Kernel Perceptron
3. **Constructing Kernels:** Understand how to build valid kernel functions
4. **Deep Dive:** Analyze the RBF kernel as an interpretable example

1 Recap

In the last lecture, we started talking about the *kernel trick*. The basic idea was that, if the original features we are given are not good enough so that a linear model isn't able to classify our data well, we can use a *feature map* $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ with $D > d$ to map $\mathbf{x} \rightarrow \phi(\mathbf{x})$ so that a model $h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$ might be powerful enough to make predictions about test points $\mathbf{x}$.

The main difficulty we ran into was that, as the number of features D grows, computing the feature map for training points $\phi(\mathbf{x}_i)$ and test points $\phi(\mathbf{x})$ grows too. If D is very large, this can even become intractable. The key fact that lets us circumvent this problem is that we can write $\mathbf{w}$ as a linear combination of our training examples with weights α :

$$\mathbf{w} = \sum_{i=1}^m \alpha_i \phi(\mathbf{x}_i)$$

Then, predictions with our model $h(\mathbf{x})$ on test points $\mathbf{x}$ become:

$$h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x})$$

On the surface, this doesn't solve our problem because we still need to compute $\phi(\mathbf{x})$. The idea of the kernel trick is that, for some feature maps ϕ , there exists a corresponding *kernel function* k so that, for any two feature vectors $\mathbf{x}, \mathbf{z}$, we have:

$$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$$

Typically, $k(\mathbf{x}, \mathbf{z})$ takes $O(d)$ time to compute for a single pair $\mathbf{x}, \mathbf{z}$. This is in stark contrast to explicitly computing the feature map $\phi(\mathbf{x})$, which can take as much as exponential time for certain feature maps! Predictions with the model now become:

$$h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i k(\mathbf{x}_i, \mathbf{x})$$

Thus, if you are given a vector $\boldsymbol{\alpha} = [\alpha_1, \dots, \alpha_m]$ corresponding to some D dimensional linear model $\mathbf{w}$, you should be able to convince yourself that you could compute the above equation in $O(md)$ time assuming $k(\mathbf{x}_i, \mathbf{x})$ takes $O(d)$ time to compute. The big idea with kernelizing machine learning algorithms is to **learn $\boldsymbol{\alpha}$ instead of $\mathbf{w}$** .

2 Kernel Machines

Now that we are equipped with the kernel trick, we can use the idea to “kernelize” algorithms. The key properties are that the algorithm accesses training points using inner products and the solution is in the span of the training points. Let us look at some examples:

2.1 Kernel Ridge Regression

Recall that ridge regression minimizes the squared loss plus an ℓ_2 regularizer:

$$\min_{\mathbf{w}} \sum_{i=1}^m (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_2^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \mathbf{w}^\top \mathbf{w}$$

An aside. While the Representer Theorem holds in general, it’s actually very easy to show that it holds for the optimal model $\mathbf{w}^*$ learned by minimizing this objective. Suppose there is an optimal $\mathbf{w}^* = \mathbf{u} + \mathbf{v}$, where $\mathbf{u} = \sum_i \alpha_i \mathbf{x}_i$ is the component of $\mathbf{w}^*$ that **can** be written as a linear combination of the $\mathbf{x}_i$ vectors, but there’s an additional component $\mathbf{v}$ that is orthogonal to each $\mathbf{x}_i$. Then:

$$\mathbf{w}^{*\top} \mathbf{x}_i = \mathbf{u}^\top \mathbf{x}_i + \underbrace{\mathbf{v}^\top \mathbf{x}_i}_{0 \text{ by definition}} = \mathbf{u}^\top \mathbf{x}_i$$

Since $\mathbf{w}^{*\top} \mathbf{x}_i = \mathbf{u}^\top \mathbf{x}_i$ for all i , clearly $\|\mathbf{X}\mathbf{w}^* - \mathbf{y}\|_2^2 = \|\mathbf{X}\mathbf{u} - \mathbf{y}\|_2^2$. However, $\|\mathbf{w}^*\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$, which means that $\|\mathbf{u}\|^2 < \|\mathbf{w}^*\|^2$. Thus $\mathbf{u}$ – which *is* a linear combination of the training features, obtains the same squared loss value but a lower L2 regularization value, so achieves a better overall objective than $\mathbf{w}^*$, which contradicts $\mathbf{w}^*$ being optimal.

The kernelized loss. With that out of the way, we can now apply the kernel trick to ridge regression. Using our replacement $\mathbf{w} \rightarrow \sum_j \alpha_j \mathbf{x}_j$, the ridge regression loss becomes:

$$\min_{\alpha} \sum_{i=1}^m \left(\underbrace{\sum_{j=1}^m \alpha_j \mathbf{x}_j^\top \mathbf{x}_i}_{\mathbf{w} \in \mathbb{R}^d} - y_i \right)^2 + \lambda \underbrace{\sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \mathbf{x}_i^\top \mathbf{x}_j}_{\mathbf{w}^\top \mathbf{w}}$$

Here, we are now minimizing over α rather than $\mathbf{w}$ directly. Clearly, these are the same thing: if there exists a α so that $\mathbf{w}^* = \sum_i \alpha_i \mathbf{x}_i$, clearly finding α^* is as good as finding $\mathbf{w}^*$. Now, we apply our feature map $\mathbf{x} \rightarrow \phi(\mathbf{x})$ and do learning in that space instead:

$$\min_{\alpha} \sum_{i=1}^m \left(\underbrace{\sum_{j=1}^m \alpha_j \phi(\mathbf{x}_j)^\top \phi(\mathbf{x}_i)}_{\mathbf{w} \in \mathbb{R}^D} - y_i \right)^2 + \lambda \underbrace{\sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)}_{\mathbf{w}^\top \mathbf{w}}$$

Finally, we apply the **kernel trick**, and replace the inner products $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = k(\mathbf{x}_i, \mathbf{x}_j)$ everywhere we see them.

$$\min_{\alpha} \sum_{i=1}^m \left(\sum_{j=1}^m \alpha_j k(\mathbf{x}_j, \mathbf{x}_i) - y_i \right)^2 + \lambda \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

All of our exponential time feature maps have been replaced by cheap to compute kernel functions!

The matrix version. A cleaner version of the loss can be derived using the matrix expansion $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i = \mathbf{X}^\top \alpha$:

$$\begin{aligned} \min_{\alpha} \|\underbrace{\mathbf{X} \underbrace{\mathbf{X}^\top \alpha}_{\mathbf{w}}}_{\mathbf{w}} - \mathbf{y}\|_2^2 + \lambda (\underbrace{\mathbf{X}^\top \alpha}_{\mathbf{w}})^\top \underbrace{\mathbf{X}^\top \alpha}_{\mathbf{w}} \\ = \min_{\alpha} \|\mathbf{X} \mathbf{X}^\top \alpha - \mathbf{y}\|_2^2 + \lambda \alpha^\top \mathbf{X} \mathbf{X}^\top \alpha \end{aligned}$$

Observe that the matrix $\mathbf{X} \mathbf{X}^\top$ is just the inner product matrix, where $[\mathbf{X} \mathbf{X}^\top]_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$. What we do is replace this with the **kernel matrix** $\mathbf{K}$ where $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$:

$$\min_{\alpha} \|\mathbf{K} \alpha - \mathbf{y}\|_2^2 + \alpha^\top \mathbf{K} \alpha$$

Taking the derivative of the above with respect to α and setting equal to zero, we solve:

$$\begin{aligned} \overbrace{\nabla_{\alpha} \|\mathbf{K} \alpha - \mathbf{y}\|_2^2 + \alpha^\top \mathbf{K} \alpha} \\ 2\mathbf{K}(\mathbf{K} \alpha - \mathbf{y}) + 2\lambda \mathbf{K} \alpha &= 0 \\ \mathbf{K}^2 \alpha - \mathbf{K} \mathbf{y} + \lambda \mathbf{K} \alpha &= 0 \\ \mathbf{K}^2 \alpha + \lambda \mathbf{K} \alpha &= \mathbf{K} \mathbf{y} \\ \mathbf{K}(\mathbf{K} + \lambda \mathbf{I}) \alpha &= \mathbf{K} \mathbf{y} \\ (\mathbf{K} + \lambda \mathbf{I}) \alpha &= \mathbf{y} \\ \alpha &= (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y} \end{aligned}$$

Therefore, the optimal value of α for kernel ridge regression is given by $(\mathbf{K} + \lambda I)^{-1} \mathbf{y}$. Put another way, the optimal linear model $\mathbf{w}_*$ in the feature space is given by:

$$\mathbf{w}_* = \Phi^\top \alpha_* = \Phi^\top (\mathbf{K} + \lambda I)^{-1} \mathbf{y}$$

where $\Phi = [\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_m)]^\top \in \mathbb{R}^{m \times D}$ is the feature matrix. Note that we never need to compute $\mathbf{w}_*$ explicitly since predictions can be made directly using the kernel:

$$h(\mathbf{x}) = \mathbf{w}_*^\top \phi(\mathbf{x}) = \mathbf{y}^\top (\mathbf{K} + \lambda I)^{-1} \mathbf{k}_{\mathbf{X}, \mathbf{x}}$$

Connection to standard ridge regression. It's instructive to see how this solution relates to the standard ridge regression closed form. Recall that for standard ridge regression, the optimal solution is:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}$$

Now let's substitute our optimal $\alpha^* = (\mathbf{X}\mathbf{X}^\top + \lambda I)^{-1} \mathbf{y}$ into $\mathbf{w}^* = \mathbf{X}^\top \alpha^*$:

$$\begin{aligned} \mathbf{w}^* &= \mathbf{X}^\top \alpha^* \\ &= \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda I)^{-1} \mathbf{y} \\ &= (\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y} \quad (\text{by the Woodbury matrix identity, see below}) \end{aligned}$$

This matches the standard ridge regression solution! This shows that kernel ridge regression with the linear kernel ($k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$) is exactly equivalent to standard ridge regression.

The Woodbury Matrix Identity

The Woodbury matrix identity has a particularly simple form when $\mathbf{C} = I$. For matrices $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \in \mathbb{R}^{n \times k}$, and $\mathbf{V} \in \mathbb{R}^{k \times n}$:

$$(\mathbf{A} + \mathbf{U}\mathbf{V})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{U} (\mathbf{I} + \mathbf{V} \mathbf{A}^{-1} \mathbf{U})^{-1} \mathbf{V} \mathbf{A}^{-1}$$

For kernel ridge regression, we apply this with $\mathbf{A} = \lambda I$, $\mathbf{U} = \mathbf{X}$, and $\mathbf{V} = \mathbf{X}^\top$, giving:

$$(\lambda I + \mathbf{X}\mathbf{X}^\top)^{-1} = \frac{1}{\lambda} (I - \mathbf{X}(\lambda I + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top)$$

Note that $\mathbf{X}\mathbf{X}^\top$ is an $m \times m$ matrix (kernel matrix) while $\mathbf{X}^\top \mathbf{X}$ is a $d \times d$ matrix (feature covariance). The identity tells us we can compute the inverse using whichever dimension is smaller. When $m \gg d$ (many samples, few features), computing $(\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1}$ is much cheaper than $(\mathbf{X}\mathbf{X}^\top + \lambda I)^{-1}$, and vice versa when $d \gg m$ (many features, few samples).

Making predictions. After training a model like this, making predictions is pretty easy:

$$h(\mathbf{x}) = \mathbf{w}^{*\top} \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i^* \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i^* k(\mathbf{x}_i, \mathbf{x})$$

This is just a linear combination of easy to compute kernel values. For a matrix form, denote by $\mathbf{k}_{\mathbf{X}, \mathbf{x}} = [k(\mathbf{x}_1, \mathbf{x}), \dots, k(\mathbf{x}_m, \mathbf{x})]$. Then:

$$h(\mathbf{z}) = \mathbf{k}_{\mathbf{X}, \mathbf{z}}^\top \alpha$$

Single step kernel ridge regression. Since we have a closed form solution for the optimal α , we can plug this in to get optimal kernel ridge regression predictions using the kernel $k(\cdot, \cdot)$:

$$h(\mathbf{z}) = \mathbf{k}_{\mathbf{X}, \mathbf{z}}^\top (\mathbf{K} + \lambda I)^{-1} \mathbf{y}$$

2.2 Kernel SVM

Let's apply kernelization to another machine learning algorithm we've learned. To kernelize support vector machines, there's broadly speaking two ways to do so:

1. Compute the *dual optimization problem*, which immediately only accesses data in terms of inner products and kernelize it.
2. Directly plug our rule $\mathbf{w} \rightarrow \sum_{i=1}^m \alpha_i \phi(\mathbf{x}_i)$ into the hinge loss + L2 regularizer.

In this class, we won't be doing the first. We mention it because *most* sources that teach kernel SVMs do it using the dual. However, an old paper (Chapelle et al., 2007) points out that the primal can be kernelized just as well, and the machinery of the dual is honestly not very useful anymore. It used to be the basis for fast, specialized SVM solvers but these solvers would no longer be particularly fast because they make poor utilization of GPU hardware.

To kernelize the original SVM loss, therefore, let's start with the original loss function using the hinge loss and ℓ_2 regularizer:

$$\mathcal{L}(\mathbf{w}) = C \sum_{i=1}^m \max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i) + \|\mathbf{w}\|_2^2$$

Next, we replace $\mathbf{x}_i$ with our feature map version $\phi(\mathbf{x}_i)$ and plug in our replacement for $\mathbf{w}$ (using a new summation variable j because we've already used i):

$$\mathcal{L}(\alpha) = C \sum_{i=1}^m \max \left(0, 1 - y_i \sum_{j=1}^m \alpha_j \phi(\mathbf{x}_j)^\top \phi(\mathbf{x}_i) \right) + \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \phi(\mathbf{x}_j)^\top \phi(\mathbf{x}_i)$$

For the regularizer, we've used the fact that $\|\mathbf{w}\|_2^2 = \mathbf{w}^\top \mathbf{w} = (\sum_{i=1}^m \alpha_i \phi(\mathbf{x}_i))^\top (\sum_{j=1}^m \alpha_j \phi(\mathbf{x}_j))$.

Next, we replace $\phi(\mathbf{x}_j)^\top \phi(\mathbf{x}_i)$ with our kernel function $k(\mathbf{x}_j, \mathbf{x}_i) = k(\mathbf{x}_i, \mathbf{x}_j)$ everywhere we see it in the loss:

$$\mathcal{L}(\alpha) = C \sum_{i=1}^m \max \left(0, 1 - y_i \sum_{j=1}^m \alpha_j k(\mathbf{x}_i, \mathbf{x}_j) \right) + \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

The above is pretty much it. Now we do gradient descent with respect to α instead of $\mathbf{w}$ and we're done. The matrix form can be written as:

$$\mathcal{L}(\alpha) = C \sum_{i=1}^m \max(0, 1 - y_i \mathbf{k}_{\mathbf{X}, \mathbf{x}_i}^\top \alpha) + \alpha^\top \mathbf{K} \alpha$$

2.3 Kernel Perceptron

Recall the Perceptron algorithm:

Algorithm 1: Perceptron

```
Initialize  $w_1 = 0 \in \mathbb{R}^d$ 
for  $t = 1, 2, \dots$  do
    if  $\exists i \in \{1, \dots, m\}$  s.t.  $y_i \neq \text{sgn}(w_t^\top x_i)$  then update  $w_{t+1} = w_t + y_i x_i$ 
    else output  $w_t$ 
end
```

Note that the updates are always in the space spanned by $\{x_1, \dots, x_m\}$ therefore we have that the optimal $w_* = \sum_{i=1}^m \alpha_i x_i$ for some $\alpha \in \mathbb{R}^m$. Using this, we can rewrite the algorithm in terms of α . *Work this out for yourself.*

Algorithm 2: Perceptron - Dual

```
Initialize  $\alpha_1 = 0 \in \mathbb{R}^d$ 
for  $t = 1, 2, \dots$  do
    if  $\exists i \in \{1, \dots, m\}$  s.t.  $y_i \neq \text{sgn}\left(\sum_{j=1}^m \alpha_{tj} \mathbf{x}_j^\top \mathbf{x}_i\right)$  then update  $\alpha_{(t+1)i} = \alpha_{ti} + y_i$ 
    else output  $\alpha_t$ 
end
```

Since this only depends on inner products, we can kernelize it straightforwardly by replacing $\mathbf{x}_j^\top \mathbf{x}_i$ with $k(\mathbf{x}_j, \mathbf{x}_i)$. The kernelized version makes predictions as:

$$h(\mathbf{x}) = \text{sgn} \left(\sum_{j=1}^m \alpha_j k(\mathbf{x}_j, \mathbf{x}) \right)$$

This has an intuitive interpretation: each training point $\mathbf{x}_j$ contributes to the prediction based on how similar it is to the test point $\mathbf{x}$ (measured by the kernel), weighted by its coefficient α_j which accumulates whenever that training point was misclassified. Points that were frequently misclassified during training will have larger magnitudes of α_j and thus more influence on the prediction.

3 Constructing Kernels

Now that we have some intuition for what a kernelized model actually *does*, how do we go about deciding what kernel to use? We've already seen a few kernels so far:

1. Linear kernel: $k_{\text{lin}}(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
2. Polynomial kernel: $k_{\text{poly}}(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\top \mathbf{z})^p$
3. RBF kernel: $k_{\text{RBF}}(\mathbf{x}, \mathbf{z}) = \exp \left(-\frac{\|\mathbf{x} - \mathbf{z}\|_2^2}{2\sigma^2} \right)$

In general, the polynomial and RBF kernels give us very powerful “general purpose” starting points for kernelized machine learning because they are extremely flexible. What other kind of kernel functions are there? Broadly speaking there are two properties of kernels that will let us know whether a proposed kernel function is a kernel:

1. A function $k(\mathbf{x}, \mathbf{z})$ is a kernel if and only if there exists a feature map ϕ so that $k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$ for all $\mathbf{x}, \mathbf{z}$. In other words, while we started with kernels as defined as an inner product in some feature space, the converse is also true: any kernel must have a corresponding feature map.
2. A function $k(\mathbf{x}, \mathbf{z})$ is a kernel if and only if, for any set of inputs $\mathbf{x}_1, \dots, \mathbf{x}_m$ the matrix $\mathbf{K}$ such that $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is a positive semidefinite matrix.
 - (a) A matrix $\mathbf{K}$ is positive semi-definite if and only if $\mathbf{v}^\top \mathbf{K} \mathbf{v} \geq 0$ for all $\mathbf{v} \neq 0$.
 - (b) A matrix $\mathbf{K}$ is positive semi-definite if and only if the eigenvalues are all greater than or equal to 0.

The above two properties generally let us verify whether some combination of existing kernels is a kernel. You can usually design new kernels by combining two existing kernels $k_1(\mathbf{x}, \mathbf{z})$ and $k_2(\mathbf{x}, \mathbf{z})$. One way to do this, for example, might be to assume the feature maps are $\phi_1(\mathbf{x})$ and $\phi_2(\mathbf{x})$ respectively (which we know exist from item 1 above). Then, if you can construct a new feature map corresponding to a combination of k_1 and k_2 , you’ll know the combination is also a kernel. Alternatively, you could show that, given the kernel matrices $\mathbf{K}_1$ and $\mathbf{K}_2$, the combination kernel matrix is positive definite.

Why construct kernels. The idea of constructing kernels from base kernels can be very powerful. Suppose $k_1(\mathbf{x}, \mathbf{z})$ and $k_2(\mathbf{x}, \mathbf{z})$ are kernels that measure similarity like the RBF kernel (in other words, the kernel values are high if $\mathbf{x}, \mathbf{z}$ are similar). Imagine that k_1 and k_2 measure different aspects of similarity between $\mathbf{x}$ and $\mathbf{z}$. If the max value of k_1 and k_2 are both 1 (like in the RBF kernel case), when would $k_1(\mathbf{x}, \mathbf{z}) + k_2(\mathbf{x}, \mathbf{z})$ be at least 1? What about $k_1(\mathbf{x}, \mathbf{z}) \times k_2(\mathbf{x}, \mathbf{z})$?

4 An Interpretable Example: The RBF Kernel

One of the most popular kernels used in machine learning is the **radial basis function (RBF)** or **Gaussian kernel**:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|_2^2}{2\sigma^2}\right)$$

Here, σ is another *hyperparameter* – a parameter that you’ll have to control yourself that changes the properties of the model. It’s like k in k Nearest Neighbors, or λ in the regularizers we’ve seen. We’re going to study this kernel a lot in this lecture, because this kernel highlights some of the most important intuitions about kernels.

What does the equation mean? To start with, let’s think about the definition above. Imagine for a moment that $\sigma = \frac{1}{2}$ so that the denominator disappears. $\|\mathbf{x} - \mathbf{z}\|_2^2$ is the squared distance between $\mathbf{x}$ and $\mathbf{z}$. We then take e to the power of minus this squared distance. When the distance

is 0, the kernel takes the value 1. When the distance is very large, the kernel value gets smaller and smaller. Thus, the kernel between two points $\mathbf{x}, \mathbf{z}$ approaches 1 as the points get closer together, and approaches 0 as the points get further apart.

Now let's think about what a model using this kernel looks like. Recall that, given training points $\mathbf{x}_1, \dots, \mathbf{x}_m$ and a test point $\mathbf{x}$, the kernelized version of our linear models with an RBF kernel can be written as:

$$h(\mathbf{x}) = \sum_{i=1}^m \alpha_i k_{\text{RBF}}(\mathbf{x}_i, \mathbf{x}),$$

but what does this expression actually *do*? Well, we just observed that, when using the RBF kernel, $k_{\text{RBF}}(\mathbf{x}_i, \mathbf{x})$ will be 1 (i.e., large) whenever $\mathbf{x}$ is close to the test point $\mathbf{x}_i$ and small whenever it's far from that test point. Now you can start to imagine how the above expression could be used to train a model even without an explicit training procedure. For each training point $\mathbf{x}_i$, choose an α_i based on the label y_i . For example, in classification, you might have α_i be negative if y_i is negative, and positive if y_i is positive. Then, when you make predictions from $\mathbf{x}$, the prediction will be the sum of a bunch of α_i values *influenced by how close $\mathbf{x}$ is to each training point*.

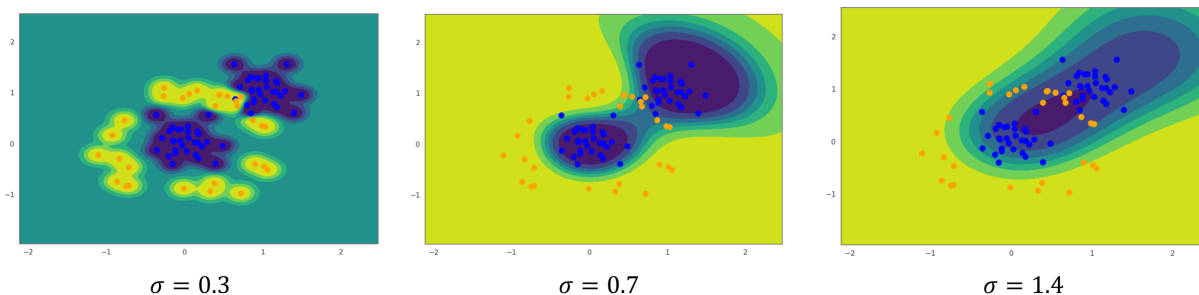


Figure 1: A model trained with an RBF kernel with three values of σ . as σ gets smaller, the model starts to look like 1 nearest neighbors. As σ grows larger, the model gets smoother because points in the space are influenced by more data.

What does σ do? σ let's us scale all of the distances. When σ is small, $\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{z}\|_2^2$ goes to zero faster than when it's large. In other words, when σ is small, $\mathbf{z}$ will have to be much closer to a training point $\mathbf{x}_i$ in order for α_i to influence the prediction. When σ is large, more α_i values will influence the prediction. Can you think of how this relates to k Nearest Neighbors?

The RBF kernel highlights one intuitive interpretation of a kernel method: for a test point $\mathbf{x}$ and each training point $\mathbf{x}_i$, compute a kernel value and make a prediction based on the α values for each $\mathbf{x}_i$. One way to interpret this with the RBF kernel is as a sort of “soft” nearest neighbors, where every data point $\mathbf{x}_i$ influences my prediction on a test point $\mathbf{x}$, and the amount of influence is determined by a similarity function k .